Mini-Project (ML for Time Series) – Soft-DTW: a Differentiable Loss Function for Time-Series – MVA 2026/2027

Pablo Borrego pabloalmendralejo@gmail.com
[TODO: second student, if any]

1 Introduction and contributions

Comparing time series point by point (Euclidean distance) is fragile: two recordings of the same phenomenon are often shifted, locally stretched or of different lengths. Dynamic Time Warping (DTW) [11] handles this by first finding the cheapest monotone *alignment* between the two series. DTW is however a minimum over alignments, hence not differentiable, and its (sub)gradient jumps whenever the optimal alignment changes. This makes it a poor training loss, and averaging or clustering under DTW has relied on heuristics such as DBA [10].

Cuturi and Blondel [4] replace the minimum by a *soft*-minimum, the same entropic smoothing that turns optimal transport into Sinkhorn distances [3]. The resulting soft-DTW is smooth, has the same $O(nm)$ cost as DTW, and its gradient is obtained by a backward pass over Bellman's recursion. The paper uses it to compute barycenters, run k-means, classify with nearest centroids and train networks that output time series.

Contributions and work repartition. **Implementation:** we implemented soft-DTW from scratch (forward and backward recursions in Numba, a PyTorch loss with the hand-written backward, and an anti-diagonal PyTorch version that runs on GPU). No soft-DTW code was reused; `tslearn` [13] is only used to load the UCR data, as the DBA baseline and as a numerical cross-check. The implementation is verified against brute-force enumeration of all alignments, finite differences and PyTorch autograd (85 unit tests). **Reproduction:** we reproduce, on 10–13 UCR datasets, the paper's barycenter (Table 1), k-means (Table 2), nearest-centroid (Fig. 7) and forecasting (Table 3) experiments. **Beyond the paper:** (i) we report magnitudes and computational costs that the paper omits, and add the baselines it does not compare to (Euclidean nearest centroid, 1-NN DTW); (ii) we show that soft-DTW is one instance of a soft dynamic program with edge costs [9], implement it generically and derive from it soft versions of amerced DTW [6], weighted DTW [7] and ERP [2], with their debiased divergences [1]; (iii) we compare this family with a neutral metric (1-NN accuracy, hyper-parameters chosen on validation). **Work repartition:** [TODO: describe the split between the two students, or remove if working alone].

2 Method

DTW and soft-DTW. Let $x \in \mathbb{R}^{n\times p}$, $y \in \mathbb{R}^{m\times p}$ be two time series and $\Delta(x,y) = [\|x_i - y_j\|^2]_{ij} \in \mathbb{R}^{n\times m}$ their cost matrix. An alignment is a binary matrix $A \in \mathcal{A}_{n,m}$ encoding a path from $(1,1)$ to (n,m) made of $\downarrow$ (x advances, y_j is repeated), $\rightarrow$ (y advances) and $\searrow$ (both advance) steps;

$|\mathcal{A}_{n,m}|$ is a Delannoy number, exponential in n, m. With the soft-minimum $\min^\gamma(a_1, \dots, a_k) = -\gamma \log \sum_i e^{-a_i/\gamma}$ (min when $\gamma = 0$),

$$\mathrm{dtw}(x, y) = \min_{A \in \mathcal{A}_{n,m}} \langle A, \Delta \rangle, \qquad \mathrm{dtw}_\gamma(x, y) = \min^\gamma_{A \in \mathcal{A}_{n,m}} \langle A, \Delta \rangle. \tag{1}$$

Because $\min^\gamma$ is a log-sum-exp, dtw_γ is smooth (infinitely differentiable) in Δ, hence in x, whereas dtw has kinks wherever the optimal path changes. Two facts used later: $\mathrm{dtw} - \gamma \log |\mathcal{A}_{n,m}| \le \mathrm{dtw}_\gamma \le \mathrm{dtw}$, so $\mathrm{dtw}_\gamma \to \mathrm{dtw}$ as $\gamma \to 0$; and $\mathrm{dtw}_\gamma(x, x) < 0$, i.e. soft-DTW is not a divergence [1].

Forward pass (Algorithm 1). Both quantities are computed by Bellman's recursion, $r_{0,0} = 0$, $r_{i,0} = r_{0,j} = +\infty$,

$$r_{i,j} = \delta_{i,j} + \min^\gamma\{r_{i-1,j-1},\, r_{i-1,j},\, r_{i,j-1}\}, \qquad \mathrm{dtw}_\gamma(x, y) = r_{n,m}, \tag{2}$$

in $O(nm)$ time; $\min^\gamma$ is evaluated with the log-sum-exp trick. The whole table $R = [r_{i,j}]$ is stored for the backward pass.

Gradient (Algorithm 2). By the chain rule, $\nabla_x \mathrm{dtw}_\gamma = (\partial\Delta/\partial x)^\top E$ with $E = \partial r_{n,m}/\partial\Delta = \mathbb{E}_\gamma[A]$, the expected alignment under the Gibbs distribution $p_\gamma(A) \propto e^{-\langle A, \Delta\rangle/\gamma}$: $e_{i,j}$ is the probability that a random path visits cell (i, j). Computing $\mathbb{E}_\gamma[A]$ directly costs $O(n^2 m^2)$; the paper instead back-propagates through the recursion. Since $r_{i,j}$ only influences its three successors,

$$e_{i,j} = \sum_{(k,l) \in \{(i+1,j),(i,j+1),(i+1,j+1)\}} e_{k,l} \, \exp\big((r_{k,l} - r_{i,j} - \delta_{k,l})/\gamma\big), \qquad e_{n,m} = 1, \tag{3}$$

where each exponential is the softmax weight of $r_{i,j}$ inside $r_{k,l}$, read from the stored table at no extra log-sum-exp cost. For the squared Euclidean cost, $\nabla_{x_i} \mathrm{dtw}_\gamma = 2 \sum_j e_{i,j}(x_i - y_j)$: each x_i is pulled towards the y_j it is likely aligned with. As $\gamma \to 0$, E tends to the optimal DTW path $A^\star$, whose discontinuous dependence on x is what soft-DTW removes (Fig. 1).

Learning with soft-DTW. The barycenter of $y_1, \dots, y_N$ solves $\min_{x \in \mathbb{R}^{n \times p}} \sum_i \frac{\lambda_i}{m_i} \mathrm{dtw}_\gamma(x, y_i)$ (Eq. 4 of the paper), a smooth but non-convex problem that we solve with L-BFGS. k-means alternates assignment and barycenter steps under dtw_γ; the nearest-centroid classifier uses one barycenter per class; and for forecasting dtw_γ is the training loss of a network (our PyTorch `autograd.Function`).

A family of soft alignment distances (our extension). Soft-DTW is one instance of a soft dynamic program on the alignment grid [9], in which each of the three edges entering cell (i, j) has its own cost:

$$r_{i,j} = \min^\gamma\{r_{i-1,j-1} + c^{\searrow}_{i,j},\, r_{i-1,j} + c^{\downarrow}_{i,j},\, r_{i,j-1} + c^{\rightarrow}_{i,j}\}. \tag{4}$$

The same backward pass returns the expected usage of every edge, from which the gradient w.r.t. x follows by the chain rule. Choosing the edge costs gives: soft-DTW ($c = \delta_{i,j}$ for all edges); *soft amerced DTW* (soft-ADTW), $c^{\downarrow} = c^{\rightarrow} = \delta_{i,j} + \omega$, a soft version of [6] where each warping step pays ω; *soft weighted DTW*, $c = w(|i - j|)\,\delta_{i,j}$ with a logistic weight [7]; and *soft ERP* [2], where non-diagonal steps match a point with a gap. For soft-ADTW, $\omega = 0$ recovers soft-DTW, $\omega \to \infty$ forbids warping (Euclidean distance for equal lengths), and $\partial r_{n,m}/\partial\omega$ is the expected number of warping steps. This matters because the entropic smoothing of soft-DTW carries a data-independent prior: for large γ, E tends to the uniform average over paths, concentrated around the diagonal whatever the data (Fig. 1, right); step costs reshape this prior. Finally, each member S can be debiased into the divergence $D(x, y) = S(x, y) - \frac{1}{2}S(x, x) - \frac{1}{2}S(y, y)$, which vanishes at $x = y$ [1].

Table 1: Datasets and diagnosis. Lag: global shift between two series of the same class (cross-correlation), in % of the length (median / 90th percentile). DTW gain: median share of the within-class Euclidean cost removed by DTW.

Dataset	Train	Test	Length	Classes	Lag % (med / p90)	DTW gain
ItalyPowerDemand	67	1029	24	2	0.0 / 8.3	0.48
SyntheticControl	300	300	60	6	3.3 / 35.2	0.76
TwoLeadECG	23	1139	82	2	2.4 / 6.1	0.90
ECG200	100	100	96	2	2.1 / 19.8	0.79
MedicalImages	381	760	99	10	1.0 / 8.2	0.78
CBF	30	900	128	3	4.7 / 25.8	0.85
ECGFiveDays	23	861	136	2	1.5 / 3.7	0.90
Plane	105	105	144	7	0.0 / 0.7	0.90
GunPoint	50	150	150	2	0.7 / 5.3	0.77
ArrowHead	36	175	251	3	0.0 / 0.0	0.85
Trace	100	100	275	4	4.7 / 14.6	0.96
Coffee	28	28	286	2	0.0 / 0.0	0.68
ShapesAll	600	600	512	60	1.2 / 27.1	0.93

3 Data

We use 13 univariate datasets of the UCR archive [5] (loaded with `tslearn`), chosen to span domains (ECG, motion, image outlines, power demand, synthetic), lengths (24 to 512) and numbers of classes (2 to 60), while keeping the quadratic cost of DTW manageable on a laptop (Table 1). The paper used a subset of 79 datasets without stating the selection criterion (and, as its Table 2 percentages imply, only 57–58 of them for k-means); our tables are a smaller but diverse replication, with 10–13 datasets depending on the cost of each experiment.

Diagnosis. All series are z-normalised (mean 0, standard deviation 1 per series), so pointwise costs $\delta_{i,j}$ are $O(1)$ and the smoothing parameter γ has a comparable meaning across datasets; this is not true for raw data, where γ should scale with the typical cost. To check whether the data actually contain the temporal distortions DTW is designed for, we drew 200 random pairs of series of the same class per dataset and measured (i) the global lag maximising their cross-correlation and (ii) the fraction of the Euclidean cost removed by DTW, $1 - \mathrm{dtw}/\|x - y\|^2$. Two regimes appear. Datasets such as CBF, Trace, SyntheticControl or ECG200 contain large global shifts (10% of the pairs are shifted by 15–35% of the length). Others (Coffee, ArrowHead, Plane) are globally aligned (lag 0), yet DTW still removes 68–90% of the Euclidean cost: the variability is made of *local* accelerations and stretches, which a global shift cannot fix but warping can. In every dataset DTW removes between 48% and 96% of the within-class Euclidean cost, which supports using an elastic geometry on all of them.

4 Results

4.1 Implementation and verification

Forward and backward passes are compiled with Numba; a PyTorch `autograd.Function` calls them, and a second, pure PyTorch version processes whole anti-diagonals at once ($n + m$ sequential steps instead of nm), which is the scheme GPU implementations use. We check the value and

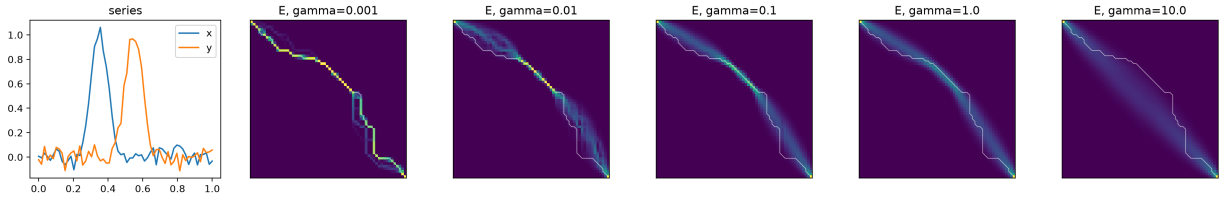


Figure 1: Expected alignment $E = \mathbb{E}_\gamma[A]$ between two shifted bumps for increasing γ; white: optimal DTW path. For $\gamma = 10$ the alignment ignores the data and concentrates around the diagonal.

Table 2: Percentage of datasets on which soft-DTW reaches a lower hard-DTW loss than DBA / the subgradient method. Ours (12 datasets for barycenters, 10 for k-means) vs. the paper (79 datasets) in parentheses.

		$\gamma = 1$	$\gamma = 0.1$	$\gamma = 0.01$	$\gamma = 0.001$
Barycenters	vs DBA, random init	58 (41)	92 (94)	100 (100)	100 (97)
	vs DBA, Euclidean init	8 (4)	25 (47)	75 (80)	100 (90)
	vs subgrad., random init	75 (96)	100 (97)	100 (97)	100 (97)
	vs subgrad., Euclidean init	17 (35)	75 (72)	92 (92)	100 (97)
k-means	vs DBA, random init	10 (16)	50 (25)	70 (60)	90 (77)
	vs DBA, Euclidean init	10 (29)	10 (24)	60 (55)	90 (69)
	vs subgrad., random init	20 (42)	60 (58)	100 (76)	90 (96)
	vs subgrad., Euclidean init	10 (46)	10 (50)	90 (66)	100 (84)

E against *brute-force enumeration of every alignment* (all Delannoy paths, $n, m \leq 5$, $\gamma \in [10^{-2}, 10]$, error $< 10^{-10}$), the gradient against finite differences, the bound $\mathrm{dtw} - \gamma \log |\mathcal{A}| \leq \mathrm{dtw}_\gamma \leq \mathrm{dtw}$, the agreement with `tslearn`, and the custom backward against autograd. All variants scale as $O(nm)$ (fitted log-log slopes 2.02–2.04), but constants differ by orders of magnitude: for $n = m = 64$ forward+backward takes 0.3 ms with Numba, 20 ms with the same code in pure Python and 0.75 s with a cell-by-cell PyTorch forward differentiated by autograd, which confirms the paper's remark that a custom backward is essential. Fig. 1 shows how E moves from the hard DTW path ($\gamma \to 0$) to a diffuse band, and for large γ to a data-independent diagonal band.

4.2 Barycenters and k-means (Tables 1–2 of the paper)

Protocol as in the paper: for each dataset, 10 random draws of 10 series of one class; barycenters from the same initialisation (random $\mathcal{N}(0, 1)$ or Euclidean mean) with at most 100 iterations; soft-DTW with L-BFGS, DBA [10] (`tslearn`) and a batch subgradient method with backtracking line search [12]. All methods are scored with the *hard* DTW objective (Eq. 4 with $\gamma = 0$), which is the objective DBA itself minimises. k-means uses k = number of classes, at most 30 Lloyd and 100 barycenter iterations, training sets capped at 100 series, 3 random initialisations (random series) and a Euclidean k-means initialisation.

Table 2 reproduces the paper's trends closely: small γ beats DBA almost always, $\gamma = 1$ rarely does, and the gap is larger from a random start. The paper only reports win rates; magnitudes and costs nuance the picture. *Barycenters:* from a random start the median soft-DTW/DBA loss ratio is 0.86 ($\gamma = 0.01$), down to 0.26 on Plane ($\gamma = 0.1$), where DBA is trapped in a poor local minimum; from the Euclidean mean, which is already a good start, the gain is only 3–4% (median ratio 0.97). *k-means:* gains shrink further (median ratio 0.97 for $\gamma \leq 0.01$). *Cost:* with $\gamma = 0.001$ a soft-DTW barycenter takes 14× (and a k-means run 16×) the time of DBA, because the objective becomes

nearly non-smooth and L-BFGS needs many more steps. Conversely, $\gamma = 1$ gives visually smooth barycenters but losses up to $3\times$ that of DBA: smoothness and fidelity to DTW pull in opposite directions, and the hard-DTW metric only rewards the latter. Finally, the subgradient baseline is very sensitive to its step size (from a random start it stalls $30\times$ above DBA on Coffee and Trace), which inflates the win rates against it.

4.3 Nearest-centroid classification (Fig. 7 of the paper)

Following the paper, data are pooled and split 50/25/25 (3 random stratified splits) and γ is selected on validation among 15 log-spaced values in $[10^{-3}, 10]$. Soft-DTW centroids are at least as accurate as DBA centroids on 83% of the datasets (paper: 75%), with mean accuracy 0.86 vs 0.81. Two baselines the paper does not report put this in context: Euclidean centroids reach 0.77, but **1-NN DTW reaches** 0.93 and is better than soft-DTW centroids on 9 of 12 datasets. Nearest centroids trade accuracy for a much smaller model and faster prediction. Moreover, the selected γ is unstable: on CBF it ranges from 0.002 to 0.19 across splits, and on ECGFiveDays the largest value $\gamma = 10$ is always selected, i.e. there is no reliable default for γ.

4.4 Multistep-ahead forecasting (Table 3 of the paper)

As in the paper, the first 60% of each series is the input and the remaining 40% the target; a one-hidden-layer MLP (256 sigmoid units, Adam, 400 full-batch epochs, hyper-parameters not given in the paper) is trained with the Euclidean loss or with soft-DTW, from random weights or from the Euclidean-trained network, on 13 datasets.

Table 3: Average rank (1 = best) of the five models on test, over 13 datasets; the paper's values (79 datasets) in parentheses.

Test metric	Init	Euclidean	$\gamma = 1$	$\gamma = 0.1$	$\gamma = 0.01$	$\gamma = 0.001$
DTW	random	3.69 (3.46)	3.92 (3.55)	2.31 (3.33)	**2.23** (2.79)	2.85 (**1.87**)
	Euclidean	3.92 (4.21)	4.46 (3.96)	2.85 (3.42)	2.08 (2.12)	**1.69** (**1.29**)
Euclidean	random	**1.31** (**1.05**)	1.69 (2.41)	3.46 (3.42)	4.00 (4.13)	4.54 (3.99)
	Euclidean	2.77 (**1.70**)	**1.62** (2.99)	2.38 (3.38)	3.77 (3.64)	4.46 (3.29)

Table 3 reproduces the main claim: each loss wins on its own metric, and when initialised from the Euclidean network the ordering in γ is the paper's (median test DTW reduced by 19–20% for $\gamma \leq 0.01$). Two differences are informative. First, trained from scratch with $\gamma = 0.001$, the network is unstable on some datasets (test DTW $3.2\times$ that of the Euclidean model on Coffee, $2\times$ on Plane), in line with the degenerate alignments reported for soft-DTW training in [14]; the Euclidean warm start removes the problem. Second, the price of shift invariance is large in Euclidean terms: the median test Euclidean error of soft-DTW models is $1.5\times$ ($\gamma = 0.1$) to $3.4\times$ ($\gamma = 0.001$) that of the Euclidean model (Fig. 2). Fig. 2 shows a more worrying failure: on ShapesAll the soft-DTW network (random init) outputs an almost flat line with isolated spikes at the extreme values, and yet obtains a *lower* test DTW than the Euclidean network (6.6 vs 9.3). Free warping lets a whole target segment be matched to a single predicted point of the right height, so degenerate predictions are cheap under DTW — both as a training loss and as an evaluation metric. Such timing and shape errors are not penalised by soft-DTW [8]; controlling them requires a cost on warping, which motivates the next section.

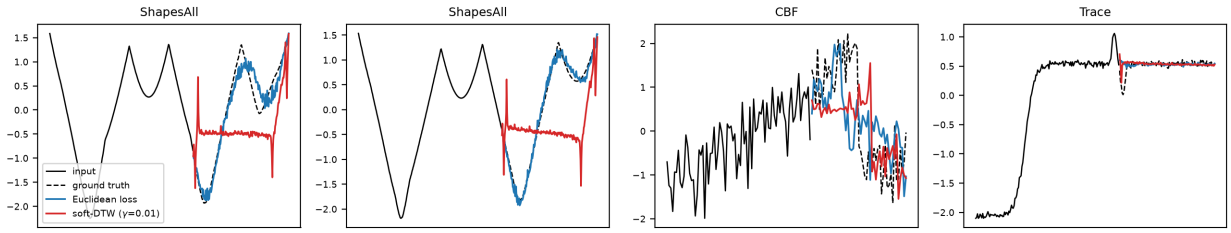


Figure 2: Test forecasts (last 40%) from MLPs trained with the Euclidean loss and with soft-DTW ($\gamma = 0.01$, random init). On ShapesAll the soft-DTW model degenerates into a flat line with spikes, which DTW scores better than the Euclidean forecast.

4.5 A family of soft alignment distances

[TODO: results of exp_family: 1-NN accuracy of soft-DTW / soft-ADTW / soft-WDTW / soft-ERP, raw vs divergence, vs Euclidean and 1D Wasserstein controls; selected gamma and omega]

5 Conclusion

We re-implemented soft-DTW from scratch, verified it against exhaustive enumeration of alignments, and reproduced the paper's barycenter, k-means and nearest-centroid results on 10–12 UCR datasets: the main claims hold, as do the trends in γ. Measuring magnitudes, costs and simple baselines shows their limits: large gains over DBA only from poor initialisations, a 10–15$\times$ computational overhead for the small γ that win, nearest centroids well below 1-NN DTW, and no stable choice of γ. [TODO: one or two sentences on the forecasting and family results.]

References

[1] Mathieu Blondel, Arthur Mensch, and Jean-Philippe Vert. Differentiable divergences between time series. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2021.

[2] Lei Chen and Raymond Ng. On the marriage of Lp-norms and edit distance. In *International Conference on Very Large Data Bases (VLDB)*, pages 792–803, 2004.

[3] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

[4] Marco Cuturi and Mathieu Blondel. Soft-DTW: a differentiable loss function for time-series. In *International Conference on Machine Learning (ICML)*, pages 894–903, 2017.

[5] Hoang Anh Dau, Eamonn Keogh, et al. The UCR time series classification archive, 2018. https://www.cs.ucr.edu/~eamonn/time_series_data_2018/.

[6] Matthieu Herrmann and Geoffrey I. Webb. Amercing: An intuitive and effective constraint for dynamic time warping. *Pattern Recognition*, 137, 2023.

[7] Young-Seon Jeong, Myong K. Jeong, and Olufemi A. Omitaomu. Weighted dynamic time warping for time series classification. *Pattern Recognition*, 44(9):2231–2240, 2011.

[8] Vincent Le Guen and Nicolas Thome. Shape and time distortion loss for training deep time series forecasting models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[9] Arthur Mensch and Mathieu Blondel. Differentiable dynamic programming for structured prediction and attention. In *International Conference on Machine Learning (ICML)*, 2018.

[10] François Petitjean, Alain Ketterlin, and Pierre Gançarski. A global averaging method for dynamic time warping, with applications to clustering. *Pattern Recognition*, 44(3):678–693, 2011.

[11] Hiroaki Sakoe and Seibi Chiba. Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1):43–49, 1978.

[12] David Schultz and Brijnesh Jain. Nonsmooth analysis and subgradient methods for averaging in dynamic time warping spaces. *Pattern Recognition*, 74:340–358, 2018.

[13] Romain Tavenard et al. Tslearn, a machine learning toolkit for time series data. *Journal of Machine Learning Research*, 21(118):1–6, 2020.

[14] Johannes Zeitler, Simon Deniffel, Michael Krause, and Meinard Müller. Stabilizing training with soft dynamic time warping: A case study for pitch class estimation with weakly aligned targets. In *International Society for Music Information Retrieval Conference (ISMIR)*, 2023.

A Additional figures

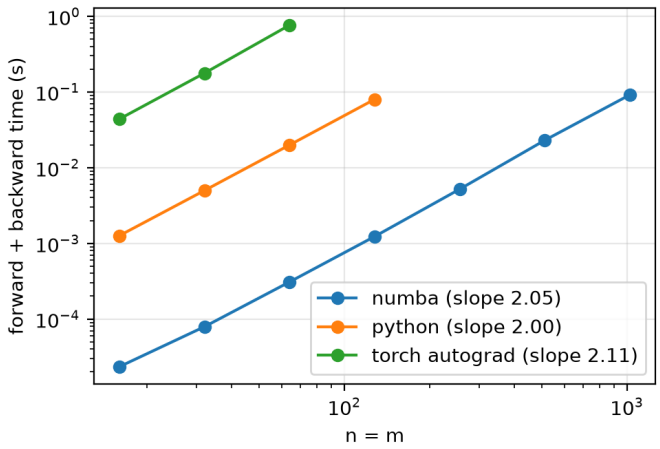


Figure 3: Forward + backward time vs. length $n = m$ (log-log). All implementations are quadratic; Numba is $\sim$65$\times$ faster than the same code in pure Python and $\sim$2400$\times$ faster than a cell-by-cell PyTorch forward differentiated by autograd ($n = 64$).

[TODO: qualitative figures: barycenters on ECG200 (paper Fig. 5), interpolation on GunPoint (Fig. 4), k-means clusters on CBF (Fig. 6), forecasts on ShapesAll (Fig. 1).]